The fault is in our starts and in ourselves: vision transformers for adversarial digit classification under a sparse data regime

Arjun Sharma
asharma3@caltech.edu

Summary

This report details my attempt to train a performant Vision Transformer (ViT) over the provided 'MNIST-in-the-wild' (MNIST-W) dataset. I achieve **72.9**% validation accuracy using a model with 5.43M parameters. The model is trained on 85% of the provided MNIST-W samples, which are passed through an aggressive data augmentation pipeline.

A DeiT-tiny architecture, inspired by [2], is used, with Shifted Patch Tokenization (SPT) and Locality Self Attention (LSA) [1]. These techniques, designed for sparse data regimes in previous literature, are shown in our study to improve model performance by up to 6%, likely due to them increasing the inductive bias in the network, which is necessary in a sparse data regime.

I include all components required by the project instructions, but swap the order of the first two parts: in section 1, I describe the data processing steps, particularly augmentations; in section 2, I provide a high-level overview of the basic training setup.

My final notebook is available here:
https://colab.research.google.com/drive/1xsVW6XLaOc29qyisI_pdJXASske06-Ze?usp=sharing.

1 Data and preprocessing

Data. The provided dataset consists of 10,294 images of digits, each of which has one of 10 labels. There are 873 ± 16 images in each class, i.e., the difference in the number of samples is at most 3.67% between any two classes. I therefore chose not to balance this dataset. All images are transformed to 128×128 later, as described in the dataloader implementation section.

Split. I chose an 85:15 train:validation split, which results in 8755 training samples and 1,539 validation samples. This split was stratified, i.e., I did not randomly selected 15% of the entire dataset directly to generate the validation dataset; rather, for each class, I selected 15% of the dataset to assemble the validation dataset. This ensured that the distribution of classes in the training and validation sets was as close to identical as possible.

Augmentation. The following augmentations are applied to every item in the training dataset, using the `transforms` module in the `TorchVision` package.

Transform	**Parameters**
Random resized crop	128×128, scale $\in [0.7, 1.0]$, ratio $\in [0.75, 1.33]$
Random augment	2 operations, magnitude 9
Elastic transform	$\alpha = 50.0$, $\sigma = 5.0$
Gaussian blur	kernel 5×5, $\sigma \sim \mathcal{U}(0.1, 2.0)$
Gaussian noise	$\epsilon \sim \mathcal{N}(0, 0.05^2)$ added per pixel
Random erasing	$p = 0.25$, scale $\in [0.02, 0.15]$
Normalization	$\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$

Table 1: Training augmentation pipeline. Note that I needed an additional 'transform' to convert the image to a PyTorch tensor before the Gaussian noise augmentation, since, up to that point, PyTorch expects a PIL image, and the last two transformations expect a tensor. Images in the validation set are passed through a different transformation pipeline: they are only resized to 128×128 and to be normalized to with the same parameters.

The values for μ and σ for the normalization steps are the values in the ImageNet dataset[1].

Then, for each item in the training dataset, I repeat it N times per epoch with independent shuffles, where $N = 2$ for my final training run. This means every unique sample is seen N times per epoch, each time with a fresh augmentation draw.

Examples of the data augmentation pipeline are presented in Figure 4.

Dataloader implementation. I use the PyTorch 'Dataset' type. My training dataset applies the transformations in Table 1 to an image loaded from a provided path (randomly sampled from the result of the stratified split function) and returns the image and its label. The validation dataset is instantiated using the same class, but with a different set of paths provided.

A PyTorch `DataLoader` is wrapped around the combined dataset for training and the validation set. A batch size of 64 is set for both. Shuffling is set to true for training and false for validation.

Augmentation analysis. Augmentation encourages invariance to transformations that do not affect digit identity, and are particularly likely to be present in MNIST-W test samples.

Given that these images were collected by different people in vastly different settings with different cameras, transformations like random resized crops and random augment operations help account for variations in framing, lighting, and camera positioning.

The elastic transform is particularly crucial for handwritten digits, as it simulates natural variations in writing style and stroke deformation without compromising digit identity.

To handle environmental differences across capture settings, Gaussian blur accounts for camera focus variations and motion blur, while Gaussian noise models sensor noise and varying lighting conditions. Random erasing improves robustness to partial occlusions that may occur in real-world scenarios (e.g., fingers partially covering the digit, shadows, or debris). The random resized crop parameters are set to a scale range of $[0.7, 1.0]$ and aspect ratio of $[0.75, 1.33]$ to preserve digit legibility while introducing realistic perspective variations.

Notably, we avoid augmentations like horizontal flipping and use elastic transforms with moderate parameters ($\alpha = 50.0$, $\sigma = 5.0$) to ensure transformations remain within the manifold of valid digit representations (e.g., preventing a 6 from appearing as a 9, or a 3 from resembling an 8).

2 Model setup

Model. More model details are in section 3. My model has 3 attention heads and contains 5,427,274 parameters.

Training. The model was trained with cross-entropy loss

$$\sum_i y^{(i)} \log(p_\theta^{(i)}) \tag{1}$$

where $p_\theta(\cdot)$ is the output of the model.

During the final training run, we end up taking an unusual approach to training. I trained iteratively for a few dozen epochs, pushing the ending of training closer to the deadline, and restarted training with a saved checkpoint multiple times while I felt like there was still scope to improve the performance of this model. This was an inefficient but realistic way of doing training under a deadline with sparse compute, where it was unclear from the loss history whether to dedicate additional time to improving results. This approach implicitly ended up mimicking a learning rate schedule with cold restarts until performance was saturated.

[1] Following advice I found online, it is acceptable to use these values for real-world images, and avoid the cost and instability of recalculating them for every training dataset – which was helpful when I was testing different amounts of augmentations.

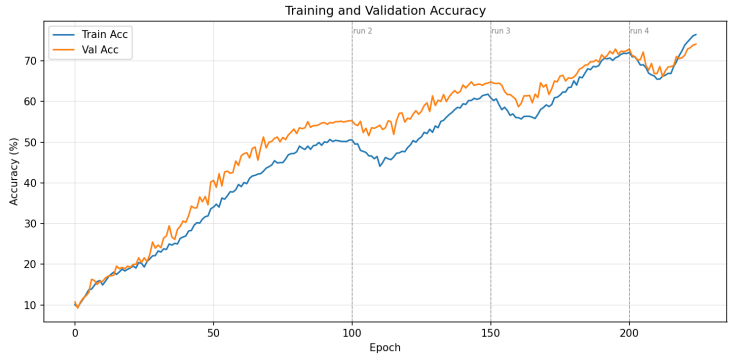


Figure 1: Accuracy over time with multiple restarted runs

Figure 2: Loss over time with multiple restarted runs

Notice that we seem to do better on the validation set for a prolonged period, due to the augmentations encouraging robust generalization, until model performance saturates with training loss and accuracy approaching their validation counterparts, where I decide to stop training.

In every run, the model was trained using the AdamW optimizer with an initial learning rate of 3×10^{-4} and weight decay set to 0.1. A batch size of 128 was used. A cosine learning rate scheduler was used in all epochs, with 10 warm-up epochs where the learning rate was linearly increased from 0 to $\mathrm{lr}_{\mathrm{base}} = 10^{-3}$, after which it decayed to $\mathrm{lr}_{\mathrm{min}} = 10^{-5} \approx 0$ following the function

$$\mathrm{lr} = \mathrm{lr}_{\mathrm{min}} + (\mathrm{lr}_{\mathrm{base}} - \mathrm{lr}_{\mathrm{min}}) \cdot 0.5(1 + \cos(p_T)) \tag{2}$$

where p_T represents the fraction of completed epochs after warmup.

It was important to use warmups during each restart after each mini-run was complete, since there was no accumulated gradient information for momentum.

Unlike in the CNN, model performance tended to climb linearly after escaping an initial low-quality parameter region which was stuck around 10% accuracy.

I additionally tested the model without SPT or LSA, which are two architectural choices we introduce, over a simple 100-epoch run with learning rate decay. This stalls at 64.839% validation accuracy.

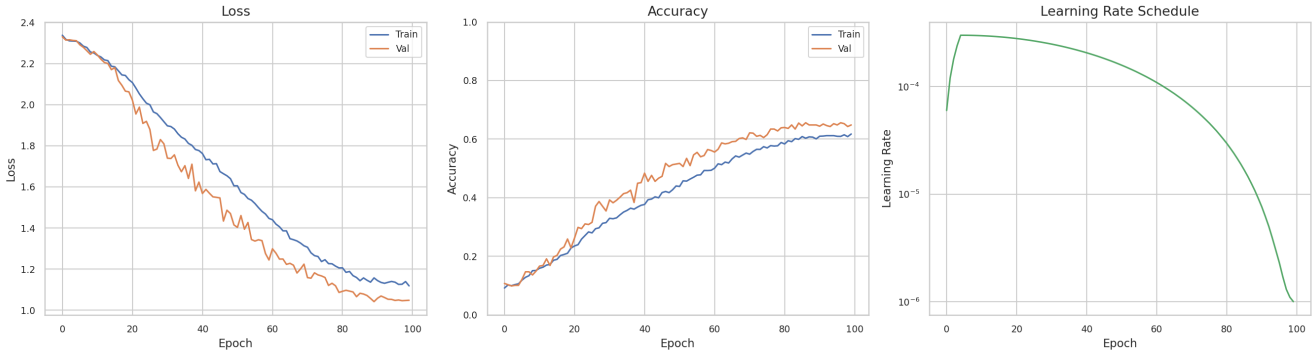


Figure 3: Model over 100 epochs without SPT or LSA. Robust generalization is achieved but model performance saturates.

I found all setups to be highly sensitive to the choice of learning rate. My initial setups did not work for a long period with learning rates of 6×10^{-4} and above, with a lot of my testing using 1×10^{-3} due to that being succesful in my previous approach. I speculate this is to that choice of LR being too high initially, which causes the parameters to bounce around a tricky optimization region. I finally concluded this since there were typically gains during the first half of the warmup period but not after, and the longer I made the warmup period, the better the model's accuracy before stalling, which suggested that the lower learning rates were helpful. Due to time constraints, I was unfortunately not able to run a comprehensive study of learning rates, especially since it was very difficult with a ViT to quickly evaluate the quality of a setup, unlike a CNN.

More details on my architecture and optimization choices are in the next section.

3 Model architecture

3.1 Interpretation

The full output of `print(model)` for the vanilla configuration is given in Appendix B. I describe each component below. Note that we use SPT and LSA following [1].

1. **Patch embedding** (`patch_embed: PatchEmbedding`). A single strided 8×8 convolution maps the 128×128 RGB image to a sequence of $N = (128/8)^2 = 256$ patch tokens, each of dimension $d = 192$. This layer plays an analogous role to the stem of a CNN: it discretises the continuous image into a fixed vocabulary of patch tokens that the transformer can process. With the SPT variant (`ShiftedPatchTokenization`), four diagonally-shifted copies of the image are concatenated channel-wise before patching (15 input channels instead of 3), so the projection becomes a `Linear(960, 192)` followed by a `LayerNorm`; this enlarges the effective receptive field of each token at embedding time without changing N.

2. **CLS token and positional embedding** (`cls_token`, `pos_embed`, `pos_drop`). A single learnable `[CLS]` vector of dimension 192 is prepended to the 256 patch tokens, giving a sequence of length 257. A learnable positional embedding of the same shape is added element-wise to encode spatial structure (transformers are otherwise permutation-invariant). `pos_drop` is a dropout layer applied after embedding; it is disabled (p=0) in the final run but retained for completeness.

3. **Transformer blocks** (`blocks: ModuleList`, $12 \times$ `TransformerBlock`). The core of the model. Each block follows the Pre-LayerNorm DeiT convention:

$$x \leftarrow x + \mathrm{DropPath}(\mathrm{Attn}(\mathrm{LN}_1(x))), \quad x \leftarrow x + \mathrm{DropPath}(\mathrm{MLP}(\mathrm{LN}_2(x))).$$

   - `norm1` / `norm2`: `LayerNorm((192,))` — normalise each token's feature vector independently before each sub-layer, stabilising training and removing the need for careful learning-rate tuning.
   - `attn: Attention` — multi-head self-attention with 3 heads (head dimension $= 192/3 = 64$). The `qkv` linear layer projects each token to queries, keys, and values simultaneously ($192 \rightarrow 576$); scaled dot-product attention is computed in parallel across heads; the `proj` linear maps the concatenated heads back to 192.

This allows every token to aggregate information from all other tokens, including the `[CLS]` token, in $O(N^2)$ time. With LSA enabled, a learnable per-head temperature scalar replaces the fixed $1/\sqrt{d_{\text{head}}}$ scaling, and a diagonal mask prevents self-attention.

- `mlp: MLP` — a two-layer position-wise feed-forward network with hidden dimension $192 \times 4 = 768$, GELU activation, and dropout. This introduces non-linearity and per-token feature transformation independent of other tokens.
- `drop_path: StochasticDepth` — randomly drops entire residual branches during training with linearly increasing probability (0 for block 0, up to 0.1 for block 11). Block 0 shows `Identity()` because its drop probability is 0; blocks 1–11 show `StochasticDepth()`.

4. **Final layer norm** (`norm: LayerNorm((192,))`). Applied to the full token sequence after all 12 blocks. This normalises representations before the classification head.

5. **Classification head** (`head: Linear(in_features=192, out_features=10)`). A linear layer that maps only the `[CLS]` token's representation to 10 class logits. This is the *only component directly responsible for classification*: it aggregates the global context accumulated by the `[CLS]` token across all 12 attention layers into a single prediction.

Low-level vs. high-level features. Unlike CNNs, transformers do not have a strict hierarchy of receptive fields imposed by convolution strides. I speculate that early blocks (0–3) encode local, low-level structure: individual patch tokens attend primarily to spatially proximate neighbors, forming representations analogous to edge detectors. Intermediate blocks (4–8) build mid-level features as tokens begin to integrate information across larger spatial extents. Later blocks (9–11) are where the `[CLS]` token aggregates global semantic information — by block 11, the `[CLS]` token has attended, through successive rounds of self-attention, to every patch in the image, producing a holistic digit representation. The classification head reads from this globally integrated `[CLS]` embedding to produce final logits.

3.2 Model capacity analysis

The base DeiT-tiny model contains **5,427,274 parameters**. Table 2 gives a component-by-component breakdown. Note that for a linear layer $\mathbb{R}^m \to \mathbb{R}^n$ the parameter count is $mn + n$.

Component	**Formula**	**Count**
Patch embedding `Conv2d(3, 192, 8, 8)`	$3 \times 192 \times 8^2 + 192$	37,056
CLS token	192	192
Positional embedding (257×192)	257×192	49,344
Per TransformerBlock:		
QKV projection `Linear(192, 576)`	$192 \times 576 + 576$	111,168
Output projection `Linear(192, 192)`	$192 \times 192 + 192$	37,056
MLP fc1 `Linear(192, 768)`	$192 \times 768 + 768$	148,224
MLP fc2 `Linear(768, 192)`	$768 \times 192 + 192$	147,648
Two LayerNorms `LN(192)` $\times$ 2	$2 \times 2 \times 192$	768
Block subtotal		444,864
12 TransformerBlocks	$12 \times 444{,}864$	5,338,368
Final LayerNorm `LN(192)`	2×192	384
CLS head `Linear(192, 10)`	$192 \times 10 + 10$	1,930
Total		**5,427,274**

Table 2: Parameter count for each component of the base DeiT-tiny model ($d = 192$, 12 blocks, 3 heads, patch size 8, image size 128×128).

The 12 transformer blocks dominate, contributing 98.4% of all parameters (5,338,368 of 5,427,274). Within each block, the MLP sub-layer is the largest component (295,872 params, 66%) because its hidden dimension is 4× the embedding dimension; the attention sub-layer (148,224 params, 33%) is smaller.

Effect of architectural extensions. SPT and LSA each add a modest number of additional parameters, summarised in Table 3. SPT replaces the patch convolution with `Linear(960, 192)` plus a `LayerNorm`, increasing the

patch-embedding count from 37,056 to 184,896 (+147,840). LSA adds one learnable temperature scalar per head per block ($12 \times 3 = 36$ parameters), a negligible change. In all cases the transformer blocks themselves are unchanged.

Configuration	**Parameters**	**Δ vs. base**
Base (no SPT / LSA)	5,427,274	—
SPT only	5,575,114	+147,840
LSA only	5,427,310	+36
SPT + LSA (final model)	5,575,150	+147,876

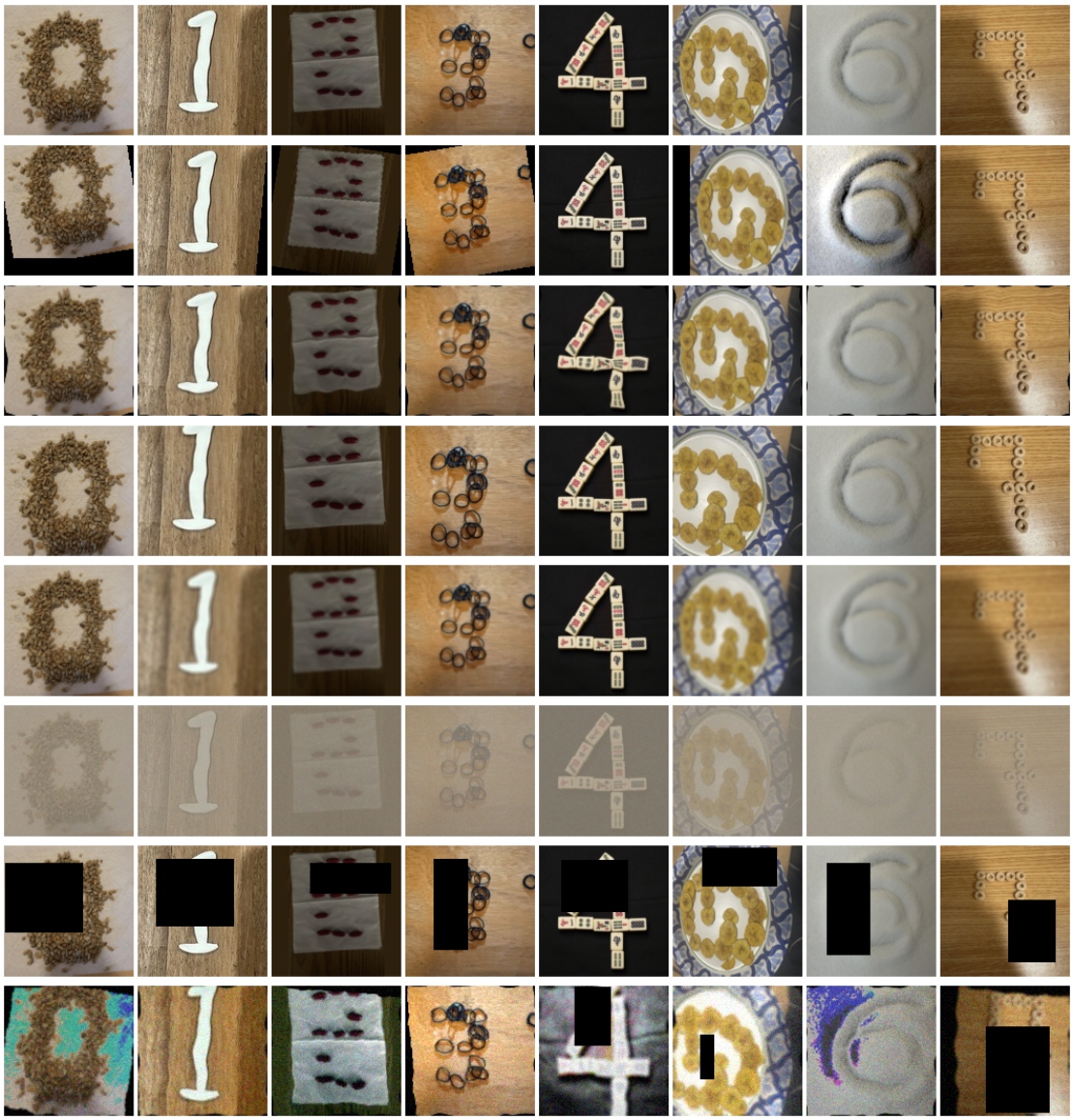
Table 3: Total trainable parameter counts for the model configurations evaluated in this project.

Could the model have been smaller or larger? The DeiT-tiny configuration (5.4M params) is already near the lower end of practical ViT scales. Given that our training set contains only $\sim 10^4$ unique images (even with repeated augmentation), a smaller model — e.g. fewer blocks or a narrower embedding — might have reduced overfitting risk; however, with aggressive augmentation and stochastic depth, overfitting was not observed to be severe in practice. Scaling up to DeiT-small (22M params, $d = 384$, 6 heads) or DeiT-base (86M, $d = 768$, 12 heads) would likely improve representational capacity, but at significantly higher memory and compute cost, and the gain may be limited by data quantity rather than model capacity.

A Augmentations used

B Output of `print(model)`

```
VisionTransformer(
  (patch_embed): PatchEmbedding(
    (proj): Conv2d(3, 192, kernel_size=(8, 8), stride=(8, 8))
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
  (blocks): ModuleList(
    (0): TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): Identity()
    )
    (1-11): 11 x TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): StochasticDepth()
    )
  )
  (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (head): Linear(in_features=192, out_features=10, bias=True)
  (dist_head): Linear(in_features=192, out_features=10, bias=True)
)
spt=False lsa=False: 5,429,204 params (5.43M)
```



(a) Original

(b) Random augmentation

(c) Elastic transform

(d) Random resized crop

(e) Gaussian Blur

(f) Gaussian noise

(g) Random erasing

(h) Full pipeline

Figure 4: Augmentation transforms applied to example images

```
  inference output: torch.Size([2, 10])
  train cls_out: torch.Size([2, 10]), dist_out: None
VisionTransformer(
  (patch_embed): ShiftedPatchTokenization(
    (proj): Linear(in_features=960, out_features=192, bias=True)
    (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
  (blocks): ModuleList(
    (0): TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): Identity()
    )
    (1-11): 11 x TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): StochasticDepth()
    )
  )
  (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (head): Linear(in_features=192, out_features=10, bias=True)
  (dist_head): Linear(in_features=192, out_features=10, bias=True)
)
spt=True lsa=False: 5,577,044 params (5.58M)
  inference output: torch.Size([2, 10])
```

```
  train cls_out: torch.Size([2, 10]), dist_out: None
VisionTransformer(
  (patch_embed): PatchEmbedding(
    (proj): Conv2d(3, 192, kernel_size=(8, 8), stride=(8, 8))
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
  (blocks): ModuleList(
    (0): TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): Identity()
    )
    (1-11): 11 x TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): StochasticDepth()
    )
  )
  (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (head): Linear(in_features=192, out_features=10, bias=True)
  (dist_head): Linear(in_features=192, out_features=10, bias=True)
)
spt=False lsa=True: 5,429,240 params (5.43M)
  inference output: torch.Size([2, 10])
  train cls_out: torch.Size([2, 10]), dist_out: None
VisionTransformer(
```

```
  (patch_embed): ShiftedPatchTokenization(
    (proj): Linear(in_features=960, out_features=192, bias=True)
    (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
  (blocks): ModuleList(
    (0): TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): Identity()
    )
    (1-11): 11 x TransformerBlock(
      (norm1): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=192, out_features=576, bias=True)
        (proj): Linear(in_features=192, out_features=192, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
      )
      (norm2): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
      (mlp): MLP(
        (net): Sequential(
          (0): Linear(in_features=192, out_features=768, bias=True)
          (1): GELU(approximate='none')
          (2): Dropout(p=0.0, inplace=False)
          (3): Linear(in_features=768, out_features=192, bias=True)
          (4): Dropout(p=0.0, inplace=False)
        )
      )
      (drop_path): StochasticDepth()
    )
  )
  (norm): LayerNorm((192,), eps=1e-05, elementwise_affine=True)
  (head): Linear(in_features=192, out_features=10, bias=True)
  (dist_head): Linear(in_features=192, out_features=10, bias=True)
)
spt=True lsa=True: 5,577,080 params (5.58M)
  inference output: torch.Size([2, 10])
  train cls_out: torch.Size([2, 10]), dist_out: None
```

C AI Usage

I used Claude code to assist in the implementation of this project, particularly setting up the full pipeline on top of the basic model and data setup, adding visualization and adding additional studies once the basic pipeline was in place. I started off by providing it a markdown file describing my plan, which I had prepared using Claude (web version). After this point, I used it to iteratively refine and change my codebase. I list my prompts below.

Prompts for Claude web:

1. For a class, I need to work on a ViT classifier of extremely adversarial data in an MNIST-style digit classification setting. Project 3 is an extension of project 2, using a ViT instead of a CNn.

   I'm attaching my report for project 2 and the guidelines for project 3. Digest and understand the nature of this setting. I will then ask you more questions.

2. Let's think through ideas on how to improve performance.

   I'm thinking we want to keep all my augmentations from the CNN project, and more - find more augmentations that are likely to work and are easy to implement (eg in the torchvision library) and justify why they would be a good idea for ViTs.

   I'm leaning towards aggressively scaling up my synthetic data. Analyze this idea and critique it if it isn't good. Notice the way I'm generating it as per my description in project 2.

   Provide further analysis on ways this project might be different from and require changes rlative to project 2

3. (I found the DeiT-tiny paper and linked it) Analyze the extent to which this would work here, considering I only have a pretrained Resnet-18 model available. If it would work, consider what architectural changes would have to be made. It seems like they propose: 1. Soft distillation loss 2. Hard distillation loss 3. Any loss with a learnable distillation token COrrect my understanding if it's wrong

   Ideally, for my project and report, I'll try to make minimal changes to the base vit architecture to account for distillation - so, for example, just add in the distillation token. this makes it nice for comparison / analysis

4. Can you give some intuition for why hard distillation works? Why is it better than just training on ground truth?

5. Do further literature review of ideas for VITs in sparse data regimes.

6. What are some other general hacks for making training more efficient? * My heuristic is that larger batches –> better for transformer. Is this correct? I'll scale up as much as possible * Should I use the muon optimizer instead of AdamW?

7. *I wrote a markdown file of instructions and uploaded them* Review these instructions for a coding agent - are they clear and comprehensive? Are there important other ideas I should include?

   Markdown file given to Claude code:

```
  You are in a directory which contains code for project 2 and project 3. Project 2 involved traini

Project 2 used Resnet-18 with an aggressive augmentation pipeline to achieve 95% accuracy. The fina



I will give you a list of features I want to implement. I will then give instructions on directory

## Features



Augmentations: for the CNN, we had
‘‘‘
def get_train_transform(img_size: int = 128):
    return transforms.Compose([
        transforms.RandomResizedCrop(img_size, scale=(0.7, 1.0), ratio=(0.75, 1.33)),
```

```
        transforms.RandomRotation(20),
        transforms.RandomPerspective(distortion_scale=0.2, p=0.5),
        transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1),
        transforms.GaussianBlur(kernel_size=5, sigma=(0.1, 2.0)),
        transforms.ToTensor(),
        transforms.Lambda(lambda x: x + torch.randn_like(x) * 0.05),
        transforms.RandomErasing(p=0.25, scale=(0.02, 0.15)),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225]),
    ])
‘‘‘

We’re going to change this to:
‘‘‘
def get_train_transform(img_size: int = 128):
    return transforms.Compose([
        transforms.RandomResizedCrop(img_size, scale=(0.7, 1.0), ratio=(0.75, 1.33)),
        transforms.RandAugment(num_ops=2, magnitude=9),
        transforms.ElasticTransform(alpha=50.0, sigma=5.0),
        transforms.GaussianBlur(kernel_size=5, sigma=(0.1, 2.0)),
        transforms.ToTensor(),
        transforms.Lambda(lambda x: x + torch.randn_like(x) * 0.05),
        transforms.RandomErasing(p=0.25, scale=(0.02, 0.33)),  # CutOut is just larger-scale erasin
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225]),
    ])
‘‘‘

Remember val transform should be just
‘‘‘
    """Deterministic transform for validation."""
    return transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225]),
    ])
‘‘‘

We’re additionally going to implement _repeated augmentation_: in every epoch, sample each image mu

Additionally apply, dependent on a CLI arg, Locality Self-Attention, which contains:
1. diagonal masking, which prevents each token from attending to itself (since self-similarity is t
2. learnable temperature scaling of the softmax, which allows the model to learn how sharply or dif

Apply, dependent on a CLI arg, Shifted Patch Tokenization. This involves taking the image, creating

Let’s keep synthetic data to 3k images in default CLI args.

### Training strategy

Learning rate warmup: 15 warmup epochs by default for 50 training epochs. Set this as an arg
Regularization:
- Stochastic depth - randomly drop entire transformer blocks. Set to 0.1
```

```
### Distillation approaches

We have a pretrained CNN model which achieves 95% accuracy in project2/checkpoints/run10_final_9k/b


1. Soft distillation
Loss = (1 - \lambda) * cross entropy loss of student against ground truth+ lambda * tau^2 KL (softm
where tau is temperature

2. Hard distillation
0.5 * cross entropy loss of student against ground truth + 0.5* cross entropy loss against hard dec

Distillation token:
> We now focus on our proposal, which is illustrated in Figure 2. We add a new token, the distillat
So when using the distillation token with hard distillation, the loss is just 0.5 CE (CLS head outp

We should design our training pipeline such that we can switch from:
1. No distillation loss
2. Soft distillation loss with lambda and tau as hyperparameters
3. Hard distillation loss
4. Hard distillation loss with the distillation token


distillation token is only used with hard distillation

## Directory setup

I want to follow a similar setup as project2/.
`src`: contains
- dataset.py
- model.py
- train.py
- pipeline.py (for final submission to HF - look at project 2 as a reference)

In model.py, set up a skeleton for the ViT, but don't fill it in yet - I want to code up the blocks

`src` also contains visualize_augmentations, visualize_embeddings, visualize_errors, visualize_pca,

A major change from project 2 is that I now want to do training in a notebook so I can use colab re

I want scripts / a notebook to be set up for the following experiments:
1. Run a vanilla ViT, with augmentations but without distillation, SPT, LSA.
2. Ablation over soft distillation, hard distillation, hard distillation with distillation token. S
3. Ablation over vanilla + SPT, vanilla + LSA, vanilla + SPT + LSA
4. Final script which will contain all the things we want - for now, assume we add in SPT, LSA, har
```

Prompts for Claude code:

1. Wait, is this implementation of stochastic depth fine? it only drops residual branches. [Pasted text #1 +21 lines]
2. Change this to use 64x64 everywhere with a 8x8 patch size.
3. Make those changes to lr - drop it to 3e-4 and warmup epochs to just 5. The weight decay fix isn't doing anything right now but there haven't been enough epochs to tell

4. Refactor the repository to get rid of all parts where plot with the tex font in matplotlib - it fails on colab

References

[1] Seung Hoon Lee, Seunghyun Lee, and Byung Cheol Song. Vision transformer for small-size datasets. *arXiv preprint arXiv:2112.13492*, 2021.

[2] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021.